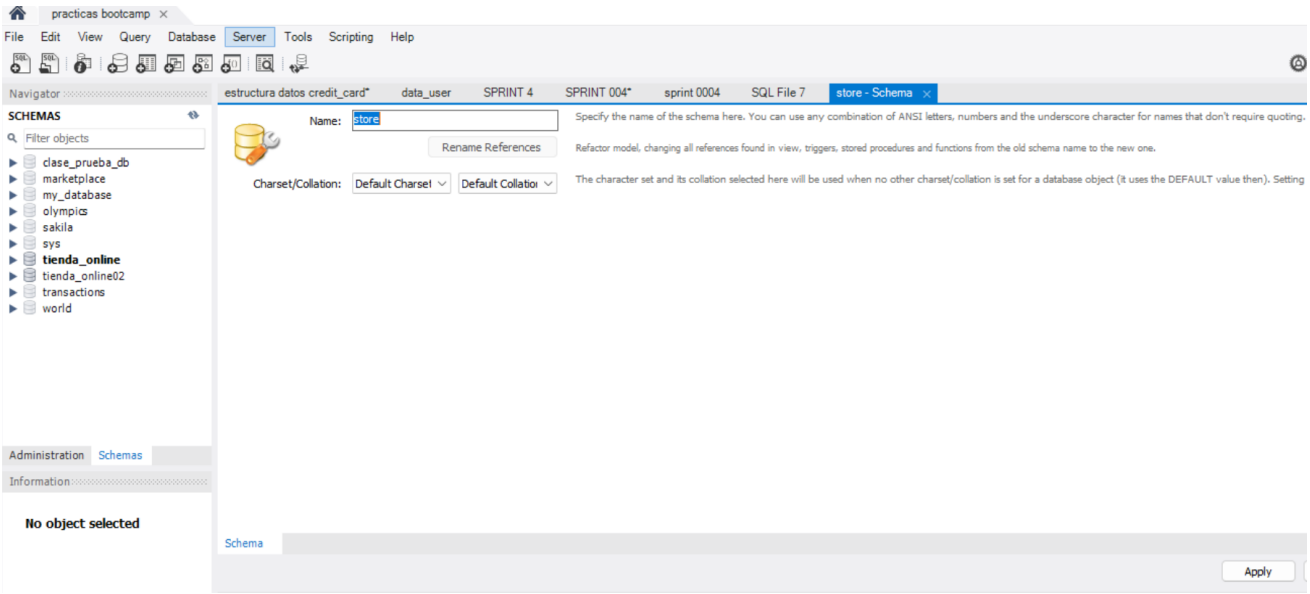
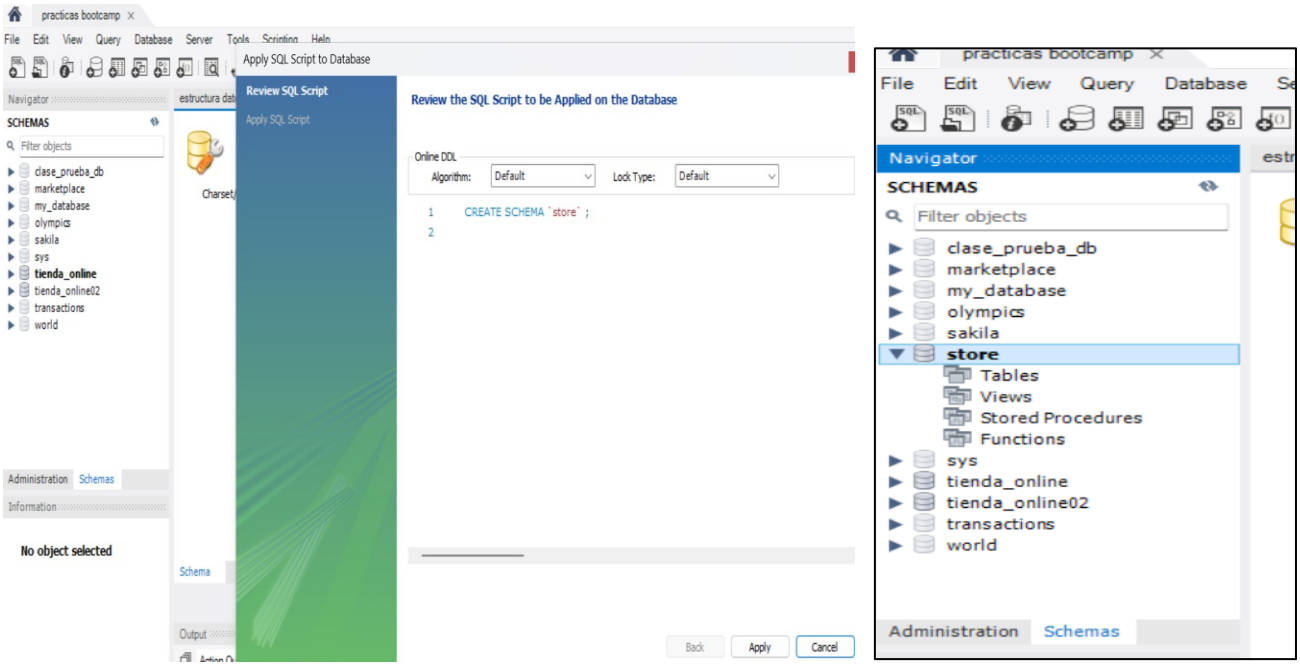
Previo a resolver los ejercicios, realicé un análisis de los datos en Excel, donde pude apreciar las tablas y determinar las PK y FK correspondientes como se aprecia en la siguiente imagen.

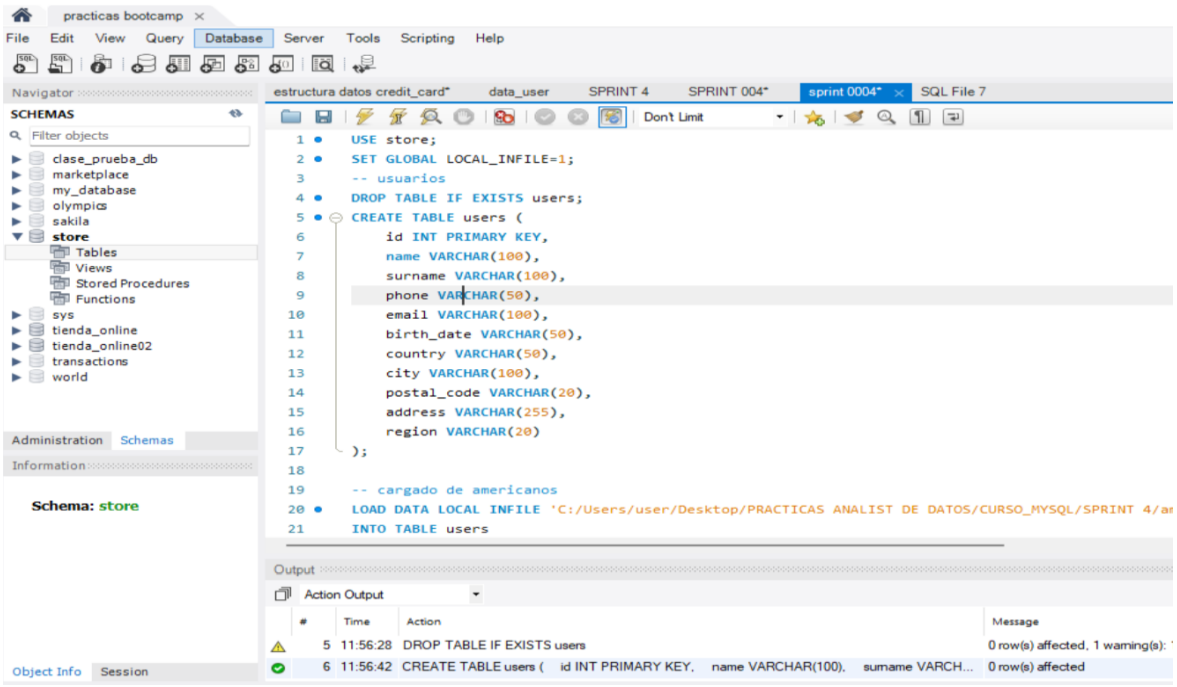
A	B	C	D	E	F	G	H	I	J	K	L
				american_users							
	id	name	surname	phone	email	birth_date	country	city	postal_code	address	
	1	Zeus	Gamble	1-282-581-0551	interdum	17-Nov-85	United State	New York	10001	348-7818 Sagittis St.	
	2	Garrett	Mcconnell	(718) 257-2412	integer.v	23-Aug-92	United State	Philadelphia	19101	903 Sit Ave	
	3	Ciaran	Harrison	(522) 598-1365	interdum	29-Apr-98	United State	Houston	77001	736-2063 Tellus St.	
				european_users							
	id	name	surname	phone	email	birth_date	country	city	postal_code	address	
	151	Meghan	Hayden	0800 746 6747	arcu.vel@	2-Jul-80	United Kingd	London	EC1A 1BB	Ap #432-4493 Aliquet Rd.	
	152	Hakeem	Alford	(0111) 367 0184	adipiscin	30-Sep-79	United Kingd	Birmingham	B1 1AA	551-8930 Lobortis Street	
	pk1										
				companies							
	company_id	company_nam	phone	email	country	website					
	b-2222	Ac Fermentum	06 85 56 52 33	donec.porttitor.tellus@	Germany	://instagram.com/site					
	b-2226	Magna A Nequ	04 14 44 64 62	risus.donec.nibh@icloud	Australia	/whatsapp.com/group/9					
	b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United St	/pinterest.com/sub/cars					
	pk2										
				credit_cards							
	id	user_id	iban	pan	pin	cvv	track1	track2	expiring_date		
	CcU-2938	275	TR301950312	5.42447E+15	3257	984	%B83837124	%B765386305	10/30/2022		
	CcU-2945	274	DO268547637	5.14242E+15	9080	887	%B4621311	%B414956843	8/24/2023		
	pk3	pk1									
				products							
	id	product_name	price	colour	weight	warehouse_id					
	1	Direwolf Stann	$161.11	#7c7c7c	1	WH-4					
	2	Tarly Stark	$9.24	#919191	2	WH-3					
	3	duel tourney L	$171.13	#d8d8d8	1.5	WH-2					
	pk4										
				transactions							
	id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	ongitude	
	CUDA7E4U-544D-47BB-	CcS-6894	b-2466	12/12/2018 8:05	161.88	0	75, 73, 98	2313	59.62050974	16.55997716	
	09456357-8E9B-475A-8257-	CcS-5135	b-2342	5/20/2024 22:49	171.13	0	3	554	45.76458842	4.843056518	
	pk5	pk3	pk2					pk1			

Posteriormente al análisis se prosiguió con la creación de la base datos con nombre "store" como se muestra en las siguientes imágenes:





La nueva base de datos podría realizarse por medio de los comandos CREATE DATABASE “store” o por el entorno grafico como se observa, ambas formas son válidas. Una vez creada la base de datos “store” continuamos realizando la estructura de tablas para introducir los datos y obtener las tablas con las trabajaremos mas adelante. En el desarrollo de la primera estructura de datos lo llamamos “users”, así cargarlo con los datos de los CSV que corresponden, no obstante, american_users y european_users cuentan con los mismos campos, por eso motivo se unió ambas tablas, agregando un campo adicional de región, para distinción cuando se unan.



```
-- cargado de americanos
LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/CURSO_MYSQL/SPRINT 4/american_users.csv'
INTO TABLE users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(
id, name, surname, phone, email,
birth_date, country, city, postal_code, address
)
SET region = 'America';

-- cargada de europeos
LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/CURSO_MYSQL/SPRINT 4/european_users.csv'
INTO TABLE users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(
id, name, surname, phone, email,
birth_date, country, city, postal_code, address
)
SET region = 'Europe';
```

#	Time	Action	Message
9	11:58:45	SET GLOBAL LOCAL_INFILE=1	0 row(s) affected
10	11:58:50	LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/CURS...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0

Como se observa dentro la base de datos creada, procedo a cargar los datos “america_users” en la estructura “users” pero incluyendo una columna para especificar que son usuarios americanos, de igual forma para “european_users” en misma estructura.

```
SET region = 'Europe';

-- limpieza
UPDATE users
SET birth_date = CASE
    WHEN birth_date LIKE '%,%'
        THEN STR_TO_DATE(birth_date, '%b %d, %Y')
    WHEN birth_date LIKE '%-%'
        THEN STR_TO_DATE(birth_date, '%d-%b-%y')
    ELSE NULL
END;
SELECT * FROM users;
```

id	name	surname	phone	email	birth_date	country	city	postal_code	address	regi
1	Zeus	Gamble	1-282-581-0551	interdum.enim@protonmail.edu	1985-11-17	United States	New York	10001	348-7818 Sagittis St.	Ame
2	Garrett	Mcconnell	(718) 257-2412	integer.vitae.nibh@protonmail.org	1992-08-23	United States	Philadelphia	19101	903 Sit Ave	Ame
3	Ciaran	Harrison	(522) 598-1365	interdum.feugiat@aol.org	1998-04-29	United States	Houston	77001	736-2063 Tellus St.	Ame
4	Howard	Stafford	1-411-740-3269	ornare.egestas@icloud.edu	1989-02-18	United States	Phoenix	85001	Ap #545-2244 Erat. Rd.	Ame
5	Hayfa	Pierce	1-554-541-2077	et.malesuada.fames@hotmail.org	1998-09-26	United States	Philadelphia	19101	341-2821 Ultrices Av.	Ame
6	Joel	Tyson	(718) 288-8020	gravida.nunc.sed@yahoo.ca	1989-10-15	United States	San Jose	95101	888-2799 Amet Street	Ame
7	Rafael	Jimenez	(817) 689-0478	eget@outlook.ca	1981-12-04	United States	Chicago	60601	8627 Malesuada Rd.	Ame

#	Time	Action	Message
12	12:10:22	UPDATE users SET birth_date = CASE WHEN birth_date LIKE '%,%' THEN STR_T...	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0
13	12:10:26	SELECT * FROM users	5000 row(s) returned

Para finalizar la tabla se realiza una limpieza de datos seguido de la verificación como se observa en la imagen anterior. Vale mencionar que la normalización de la fecha se realizo para mejor visualización, no obstante, aunque no lo requería usar una serie de combinaciones no es prudente.

Repetimos el proceso con las demás tablas, tomando en cuenta que los campos son diferentes y prestando atención a los datos.

Tabla companies

```
52            THEN STR_TO_DATE(birth_date, '%d-%b-%y')
53        ELSE NULL
54    END;
55    SELECT * FROM users;
56    -- companies
57    DROP TABLE IF EXISTS companies;
58    CREATE TABLE companies (
59        company_id VARCHAR(20) PRIMARY KEY,
60        company_name VARCHAR(100),
61        phone VARCHAR(50),
62        email VARCHAR(100),
63        country VARCHAR(50),
64        website VARCHAR(100)
65    );
66    LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/CURSO_MYSQL/SPRINT 4/companies.csv'
67    INTO TABLE companies
68    FIELDS TERMINATED BY ','
69    ENCLOSED BY '"'
70    LINES TERMINATED BY '\n'
71    IGNORE 1 ROWS;
72    -- credit_cards tabla estructura
73    DROP TABLE IF EXISTS credit_cards;
```

Output

Action Output

#	Time	Action	Message
15	12:18:53	CREATE TABLE companies (company_id VARCHAR(20) PRIMARY KEY, company_n...	0 row(s) affected
16	12:19:01	LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/C...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0

Tabla credit_cards

```sql
DROP TABLE IF EXISTS credit_cards;
CREATE TABLE credit_cards (
    id VARCHAR(20) PRIMARY KEY,
    user_id INT,
    iban VARCHAR(50),
    pan VARCHAR(50),
    pin VARCHAR(10),
    cvv VARCHAR(10),
    track1 TEXT,
    track2 TEXT,
    expiring_date VARCHAR(20),
    FOREIGN KEY (user_id) REFERENCES users(id)
);
-- carga de datos a credit cards
LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/CURSO_MYSQL/SPRINT 4/credit_cards.csv'
INTO TABLE credit_cards
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Output

Action Output

#	Time	Action	Message
22	12:29:12	DROP TABLE IF EXISTS credit_cards	0 row(s) affected
23	12:29:18	CREATE TABLE credit_cards (id VARCHAR(20) PRIMARY KEY, user_id INT, iban ...	0 row(s) affected
24	12:29:31	LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/C...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0

Table products

```sql
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;

-- produts
DROP TABLE IF EXISTS products;
CREATE TABLE products (
    id INT PRIMARY KEY,
    product_name VARCHAR(100),
    price_$ VARCHAR(20),
    colour VARCHAR(20),
    weight_kg VARCHAR(20),
    warehouse_id VARCHAR(20)
);
LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/CURSO_MYSQL/SPRINT 4/products.csv'
INTO TABLE products
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Output

Action Output

#	Time	Action	Message
30	12:34:43	CREATE TABLE products (id INT PRIMARY KEY, product_name VARCHAR(100), p...	0 row(s) affected
31	12:34:51	LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/C...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0

Tabla transactions

```sql
--    transactions
DROP TABLE IF EXISTS transactions;
CREATE TABLE transactions (
    id VARCHAR(50) PRIMARY KEY,
    card_id VARCHAR(20),
    business_id VARCHAR(20),
    timestamp VARCHAR(50),
    amount VARCHAR(50),
    declined VARCHAR(10),
    product_ids VARCHAR(255),
    user_id INT,
    lat VARCHAR(50),
    longitude VARCHAR(50),
    FOREIGN KEY (card_id) REFERENCES credit_cards(id),
    FOREIGN KEY (business_id) REFERENCES companies(company_id),
    FOREIGN KEY (user_id) REFERENCES users(id)
);
-- carga de datos transactions
LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/CURSO_MYSQL/SPRINT 4/transactions.csv'
INTO TABLE transactions
FIELDS TERMINATED BY ';'
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
```

Output

Action Output

#	Time	Action	Message
32	12:37:09	DROP TABLE IF EXISTS transactions	0 row(s) affected, 1 warning(s): 1051 Unknown table 'store.transactions'
33	12:37:24	CREATE TABLE transactions (id VARCHAR(50) PRIMARY KEY, card_id VARCHAR(2...	0 row(s) affected

```sql
-- carga de datos transactions
LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/CURSO_MYSQL/SPRINT 4/transactions.csv'
INTO TABLE transactions
FIELDS TERMINATED BY ';'
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
-- limpieza de trtansactions
UPDATE transactions
SET
    timestamp = CASE
        WHEN timestamp LIKE '%-%-% %:%:%'
            THEN STR_TO_DATE(timestamp, '%Y-%m-%d %H:%i:%s')
        WHEN timestamp LIKE '%/%/% %:%'
            THEN STR_TO_DATE(timestamp, '%d/%m/%Y %H:%i')
        ELSE NULL
    END,
    amount = REPLACE(amount, '$', ''),
    declined = CASE
        WHEN declined IN ('1','true','TRUE') THEN 1
        ELSE 0
    END;
SELECT * FROM transactions;
```

Output

Action Output

#	Time	Action	Message
34	12:38:34	LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/C...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0
35	12:39:06	UPDATE transactions SET timestamp = CASE WHEN timestamp LIKE '%-%-% %:%:%...	0 row(s) affected Rows matched: 100000 Changed: 0 Warnings: 0

```sql
SELECT * FROM transactions;
-- ejercicio 1 nivel 1
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude
00043A49-2949-494B-A5DD-A5BAE3BB19DD	CcS-9294	b-2458	2024-08-28 07:16:46	395.43	0	16, 26, 97, 87	4713	46.19992926158272	1.4355402821327607
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	2016-12-21 20:07:18	155.63	0	66, 69, 87	438	41.59720554463741	12.22175994259365
00045D6B-ED2E-4F2F-8186-CEE074D875D0	CcS-6699	b-2390	2020-07-14 15:37:45	326.01	0	30, 11, 16, 81	2118	29.757295899964348	-95.37963676492454
000481C3-1C26-4FEF-83A0-4CD0EB004BBD	CcS-6696	b-2230	2017-09-04 19:44:53	161.6	0	72	2115	53.54888376797025	-113.50305274646564
00051AA4-9CBE-4268-B070-C38062A1B3E2	CcS-7606	b-2266	2017-01-05 18:19:25	148.91	0	18	3025	52.20836951654172	5.690806474241335
0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	CcU-3358	b-2598	2023-09-23 04:51:43	294.59	0	35, 33, 19	215	53.553485560256014	-113.4991339874781
0009A151-9BCF-4E31-9053-A468FF77FAAB	CcS-7509	b-2546	2023-12-31 00:06:36	383.63	0	93, 55, 28, 91	2928	51.93615735576977	5.342650184655181

transactions 2

Output

Action Output

#	Time	Action	Message
35	12:39:06	UPDATE transactions SET timestamp = CASE WHEN timestamp LIKE '%-%-% %:%:%...	0 row(s) affected Rows matched: 100000 Changed: 0 Warnings: 0
36	12:40:01	SELECT * FROM transactions	100000 row(s) returned

NIVEL 1. Ejercicio 1: Realice una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas

```sql
-- ejercicio 1 nivel 1
-- Realice una subconsulta que muestre a todos los usuarios
-- con más de 80 transacciones utilizando al menos 2 tablas.
SELECT *
FROM users  u
 WHERE u.id IN(
        SELECT
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	name	surname	phone	email	birth_date	country	city	postal_code	address	region
185	Molly	Gilliam	0800 120 8023	donec@outlook.couk	1993-12-21	United Kingdom	London	EC1A 1BB	P.O. Box 202, 5638 Mi Rd.	Europe
289	Dxwgi	Hwcru	+98-309-8797	dxwgi.hwcru@example.com	1976-08-20	Germany	Stuttgart	70173	82 Hwcru Street	Europe
318	Bnyr	Astuw	+33-120-9644	bnyr.astuw@example.com	1974-05-03	Italy	Genoa	16100	53 Astuw Street	Europe
454	Sfzzoh	Xgvfridxs	+58-495-3945	sfzzoh.xgvfridxs@example.com	1962-08-28	Poland	Gdansk	80-001	52 Xgvfridxs Street	Europe

users 20

Output

Action Output

#	Time	Action	Message
116	13:46:01	DROP TABLE transaction_products	Error Code: 1051. Unknown table 'store.transaction_products'
117	13:47:49	CREATE TABLE transaction_products (transaction_id VARCHAR(100), product_id VA...	0 row(s) affected
118	13:47:57	INSERT INTO transaction_products (transaction_id, product_id) SELECT t.id, jt.produc...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0
119	13:54:36	SELECT * FROM users u WHERE u.id IN(SELECT user_id FROM transactions GRO...	4 row(s) returned

Ejercicio 2: Muestra el monto promedio del propietario de una tarjeta de crédito IBAN en Donec Ltd, utiliza al menos 2 tablas.

Nivel 2 ejercicio 1: Cree una nueva tabla que refleje el estado de las tarjetas de crédito según Si las últimas tres transacciones han sido rechazadas, entonces está inactivo; si al menos una no es rechazada, entonces está activa. Comenzando en esta tabla, responde: ¿Cuántas tarjetas están activas?

```sql
-- Muestra el monto promedio del propietario de una tarjeta de
-- crédito IBAN en Donec Ltd, utiliza al menos 2 tablas.
SELECT
    c.company_name,
    cc.iban,
   ROUND (AVG(t.amount),2) AS avg_amount
FROM transactions t
JOIN credit_cards cc ON t.card_id = cc.id
JOIN companies c ON t.business_id = c.company_id
WHERE c.company_name = 'Donec Ltd'
GROUP BY  cc.iban;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	iban	avg_amount
Donec Ltd	SK9446370242474562577506	142.96
Donec Ltd	XX7767529178459529755555640	257.37
Donec Ltd	XX4138273622897193049089 90	139.59
Donec Ltd	XX3477872460707696107803 08	240.41
Donec Ltd	XX6887684365430908948546 02	188.58
Donec Ltd	MK28368851538688349	439.39

Result 11

Output

Action Output

#	Time	Action	Message
46	13:21:31	SELECT * FROM users u WHERE u.id IN(SELECT user_id FROM transactions GRO...	4 row(s) returned
47	13:23:11	SELECT c.company_name, cc.iban, AVG(t.amount) AS avg_amount FROM transacti...	371 row(s) returned
48	13:23:59	SELECT c.company_name, cc.iban, ROUND (AVG(t.amount),2) AS avg_amount FR...	371 row(s) returned

```sql
CREATE TABLE card_status AS
WITH ranked AS (
    SELECT
        t.card_id,
        t.declined,
        t.timestamp,
        ROW_NUMBER() OVER (
            PARTITION BY t.card_id
            ORDER BY t.timestamp DESC
        ) AS rn
    FROM transactions t
),
last_3 AS (
    SELECT *
    FROM ranked
    WHERE rn <= 3
)
SELECT
    card_id,
    CASE
        WHEN SUM(declined) = 3 THEN 'inactive'
        ELSE 'active'
    END AS status
FROM last_3
GROUP BY card_id;
```

Output

Action Output

#	Time	Action	Message
37	12:43:19	SELECT u.id, u.name, u.surname, t.total_transactions FROM users u JOIN (SE...	4 row(s) returned
38	12:47:37	SELECT c.company_name, cc.iban, AVG(t.amount) AS avg_amount FROM transacti...	371 row(s) returned
39	12:51:10	CREATE TABLE card_status AS WITH ranked AS (SELECT t.card_id, t.declin...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

```sql
SELECT COUNT(*) AS active_cards
FROM card_status
WHERE status = 'active';

-- nivel 3: Crea una tabla con la que podamos adjuntar los datos del nuevo archivo products.csv
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

active_cards
4995

Result 5

Output

Action Output

#	Time	Action	Message
38	12:47:37	SELECT c.company_name, cc.iban, AVG(t.amount) AS avg_amount FROM transacti...	371 row(s) returned
39	12:51:10	CREATE TABLE card_status AS WITH ranked AS (SELECT t.card_id, t.declin...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
40	12:52:10	SELECT COUNT(*) AS active_cards FROM card_status WHERE status = 'active'	1 row(s) returned

En el Código anterior se crea una nueva tabla llamada card_status, asi mismo hago uso de una instrucción de la función ventana ROW_NUMBER, donde se toma los datos de la tabla transactions para cada tarjeta. Asi asignar “rn” a cada transacción, posteriormente paso a filtrar las ultimas 3 transacciones de las tarjetas.

Nivel 3: Crea una tabla con la que podamos adjuntar los datos del nuevo archivo products.csv con la base de datos creada, dado que desde la transacción tienes product_ids. Genera la siguiente consulta: Ejercicio 1 Necesitamos saber el número de veces que se ha vendido cada producto.

```sql
-- nivel 3: Crea una tabla con la que podamos adjuntar los datos del nuevo archivo products.csv
-- con la base de datos creada, dado que desde la transacción tienes product_ids. Genera la siguiente consulta:
-- Ejercicio 1: Necesitamos saber el número de veces que se ha vendido cada producto.
CREATE TABLE transaction_products (
    transaction_id VARCHAR(50),
    product_id INT,
    PRIMARY KEY (transaction_id, product_id)
);
INSERT INTO transaction_products (transaction_id, product_id)
SELECT
    t.id,
    jt.product_id
FROM transactions t,
JSON_TABLE(
    CONCAT('[', t.product_ids, ']'),
    "$[*]" COLUMNS(product_id INT PATH "$")
) AS jt;
SELECT
```

Output

Action Output

#	Time	Action	Message
41	12:58:54	CREATE TABLE transaction_products (transaction_id VARCHAR(50), product_id INT, ...	0 row(s) affected
42	12:58:58	INSERT INTO transaction_products (transaction_id, product_id) SELECT t.id, jt.produc...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0

```sql
SELECT
    p.id,
    p.product_name,
    COUNT(tp.product_id) AS times_sold
FROM transaction_products tp
JOIN products p ON p.id = tp.product_id
GROUP BY p.id, p.product_name
ORDER BY times_sold DESC;
```

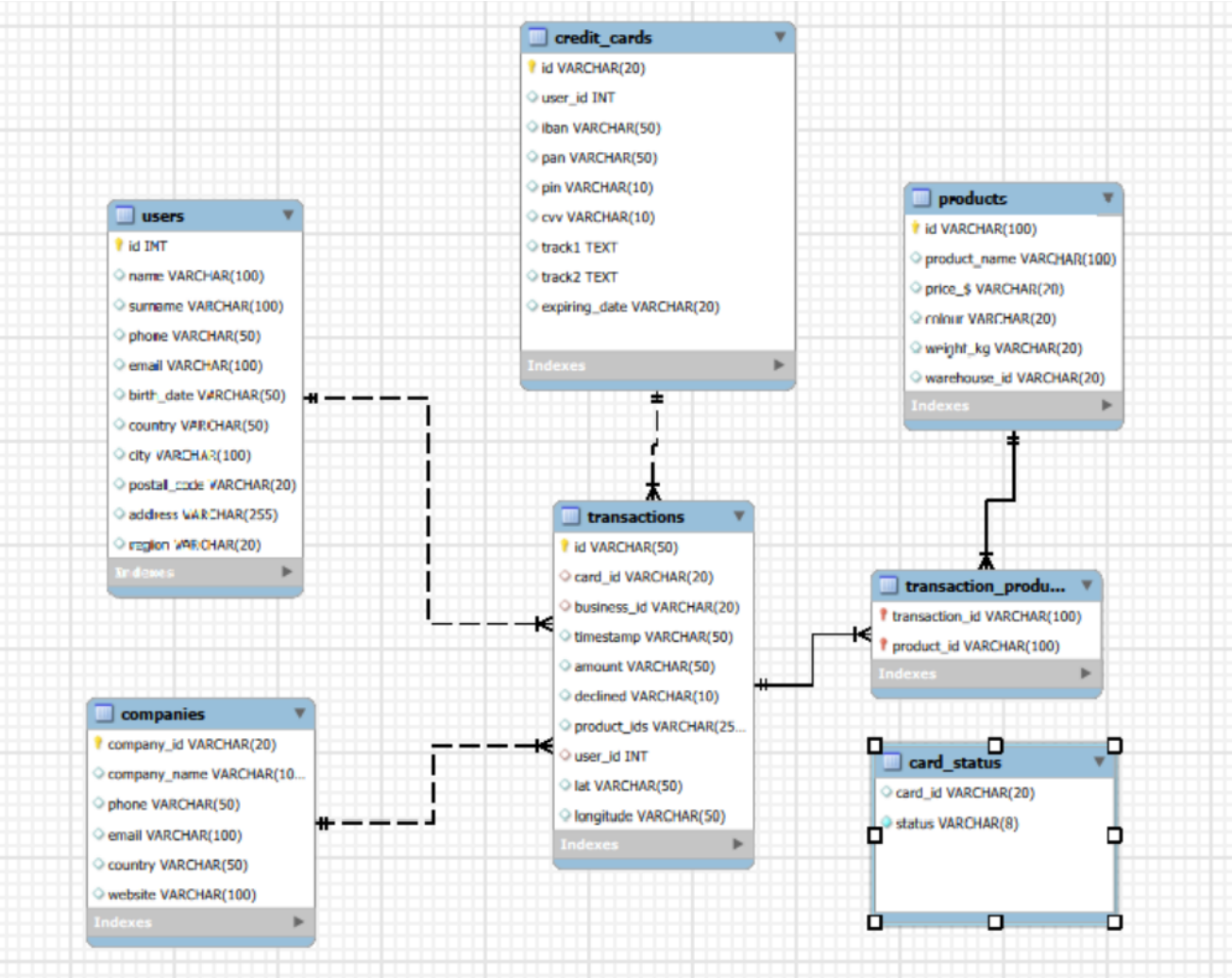
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	product_name	times_sold
52	riverlands the duel	2654
29	Tully maester Tarly	2635
21	duel Direwolf	2609
16	the duel warden	2608
66	mustafar jinn	2601
87	sith Jade	2598
33	duel warden	2597

Result 7

Output

Action Output

#	Time	Action	Message
42	12:58:58	INSERT INTO transaction_products (transaction_id, product_id) SELECT t.id, jt.produc...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0
43	12:59:09	SELECT p.id, p.product_name, COUNT(tp.product_id) AS times_sold FROM transacti...	100 row(s) returned
44	13:00:49	SELECT p.id, p.product_name, COUNT(tp.product_id) AS times_sold FROM transacti...	100 row(s) returned

Es preciso crear una tabla intermedia por medio de json, por tanto, se crea una tabla llamada transaction_products como tabla relacional, pero como un paso intermedio, cada fila representa un producto dentro de una transacción. Este código transforma datos almacenados en formato JSON dentro de una tabla en una estructura relacional, para analizarlos mejor; entonces convierte listas de productos por transacción en filas individuales y calcula la frecuencia de venta de cada producto.